

本节内容

# 图的存储

## 邻接矩阵法

# 知识总览

## 图的存储

邻接矩阵

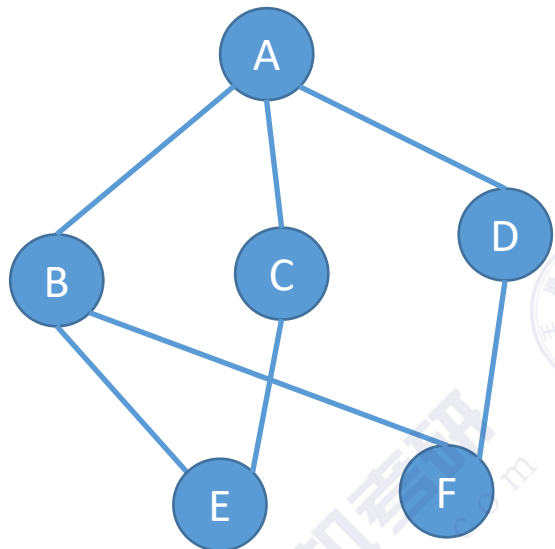
邻接表

十字链表

邻接多重表

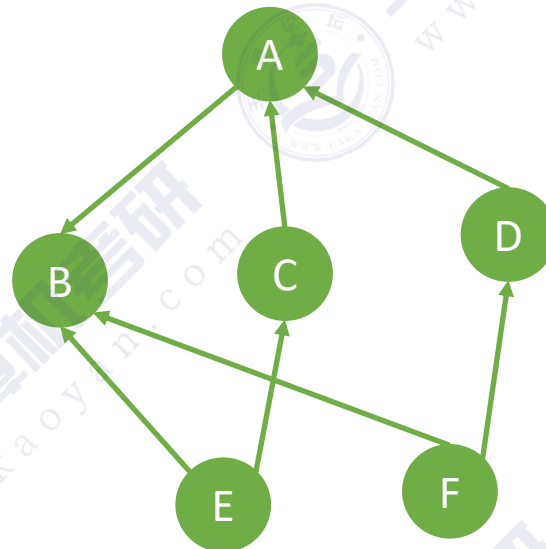
## 图的存储——邻接矩阵法

无向图



|   | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A | 0 | 1 | 1 | 1 | 0 | 0 |
| B | 1 | 0 | 0 | 0 | 1 | 1 |
| C | 1 | 0 | 0 | 0 | 1 | 0 |
| D | 1 | 0 | 0 | 0 | 0 | 1 |
| E | 0 | 1 | 1 | 0 | 0 | 0 |
| F | 0 | 1 | 0 | 1 | 0 | 0 |

有向图



|   | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A | 0 | 1 | 0 | 0 | 0 | 0 |
| B | 0 | 0 | 0 | 0 | 0 | 0 |
| C | 1 | 0 | 0 | 0 | 0 | 0 |
| D | 1 | 0 | 0 | 0 | 0 | 0 |
| E | 0 | 1 | 1 | 0 | 0 | 0 |
| F | 0 | 1 | 0 | 1 | 0 | 0 |

顶点中可以存  
更复杂的信息

可以用 bool 型  
或枚举型变量  
表示边

```
#define MaxVertexNum 100
```

```
typedef struct{
```

```
    char Vex[MaxVertexNum];
```

```
    int Edge[MaxVertexNum][MaxVertexNum];
```

```
    int vexnum, arcnum;
```

```
} MGraph;
```

//顶点数目的最大值

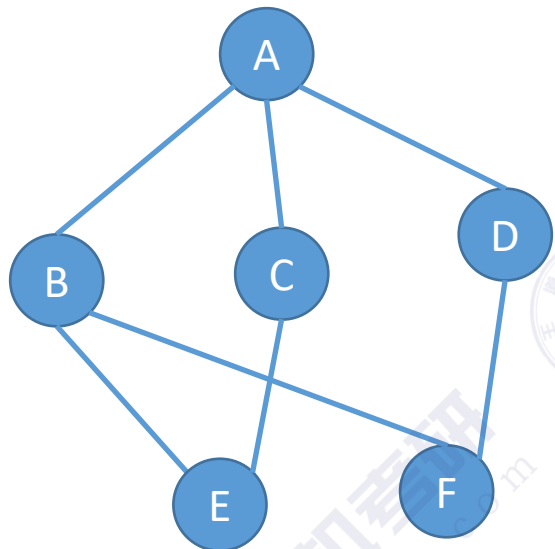
//顶点表

//邻接矩阵, 边表

//图的当前顶点数和边数/弧数

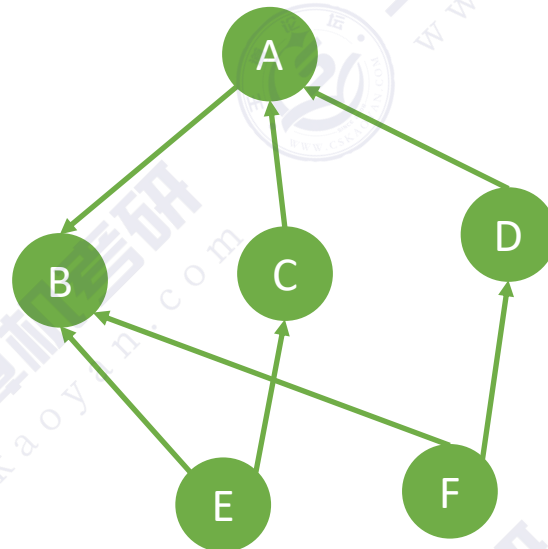
## 图的存储——邻接矩阵法

无向图



|   | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A | 0 | 1 | 1 | 1 | 0 | 0 |
| B | 1 | 0 | 0 | 0 | 1 | 1 |
| C | 1 | 0 | 0 | 0 | 1 | 0 |
| D | 1 | 0 | 0 | 0 | 0 | 1 |
| E | 0 | 1 | 1 | 0 | 0 | 0 |
| F | 0 | 1 | 0 | 1 | 0 | 0 |

有向图



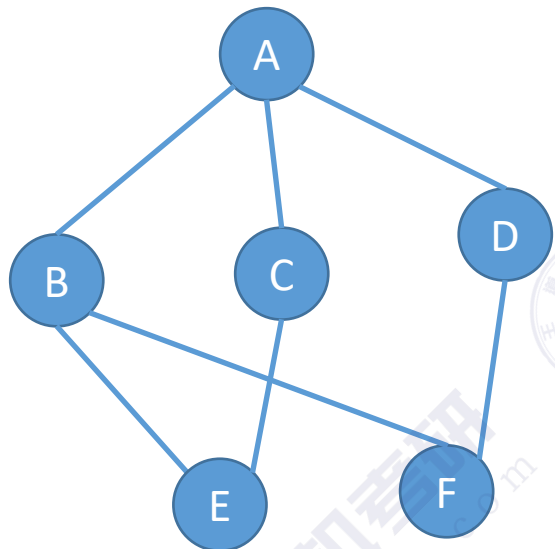
|   | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A | 0 | 1 | 0 | 0 | 0 | 0 |
| B | 0 | 0 | 0 | 0 | 0 | 0 |
| C | 1 | 0 | 0 | 0 | 0 | 0 |
| D | 1 | 0 | 0 | 0 | 0 | 0 |
| E | 0 | 1 | 1 | 0 | 0 | 0 |
| F | 0 | 1 | 0 | 1 | 0 | 0 |

结点数为 $n$ 的图 $G=(V, E)$ 的邻接矩阵 $A$ 是 $n \times n$ 的。将 $G$ 的顶点编号为 $v_1, v_2, \dots, v_n$ ，则

$$A[i][j] = \begin{cases} 1, & \text{若}(v_i, v_j) \text{或} \langle v_i, v_j \rangle \text{是} E(G) \text{中的边} \\ 0, & \text{若}(v_i, v_j) \text{或} \langle v_i, v_j \rangle \text{不是} E(G) \text{中的边} \end{cases}$$

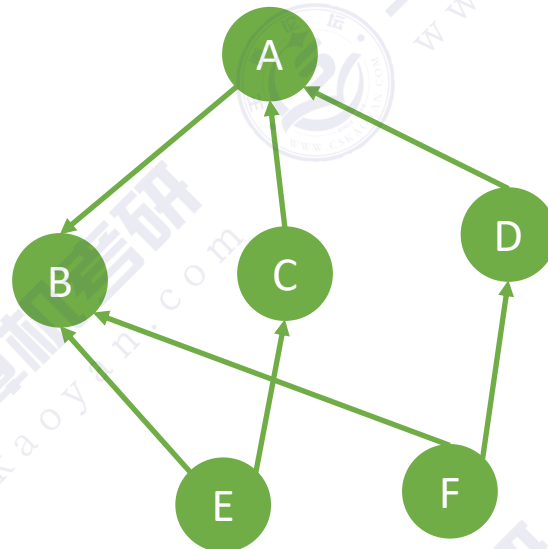
## 图的存储——邻接矩阵法

无向图



|   | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A | 0 | 1 | 1 | 1 | 0 | 0 |
| B | 1 | 0 | 0 | 0 | 1 | 1 |
| C | 1 | 0 | 0 | 0 | 1 | 0 |
| D | 1 | 0 | 0 | 0 | 0 | 1 |
| E | 0 | 1 | 1 | 0 | 0 | 0 |
| F | 0 | 1 | 0 | 1 | 0 | 0 |

有向图



|   | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A | 0 | 1 | 0 | 0 | 0 | 0 |
| B | 0 | 0 | 0 | 0 | 0 | 0 |
| C | 1 | 0 | 0 | 0 | 0 | 0 |
| D | 1 | 0 | 0 | 0 | 0 | 0 |
| E | 0 | 1 | 1 | 0 | 0 | 0 |
| F | 0 | 1 | 0 | 1 | 0 | 0 |

第*i*个结点的度 = 第*i*行（或第*i*列）的非零元素个数

第*i*个结点的出度 = 第*i*行的非零元素个数  
第*i*个结点的入度 = 第*i*列的非零元素个数  
第*i*个结点的度 = 第*i*行、第*i*列的非零元素个数之和

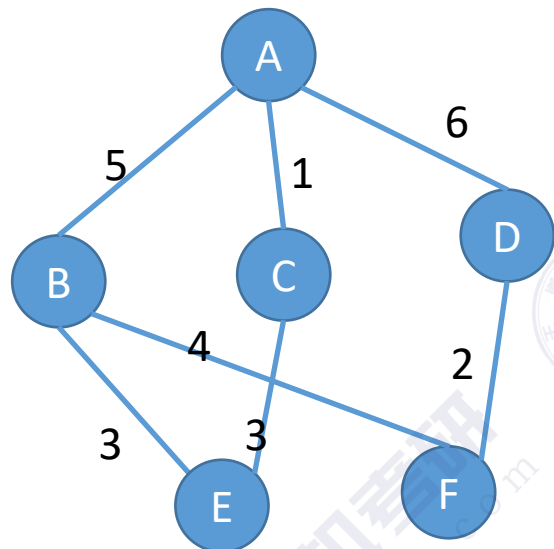
思考：如何求顶点的度、入度、出度？

如何找到与一个顶点相连的边/弧？

邻接矩阵法求顶点的度/出度/入度的时间复杂度为  $O(|V|)$

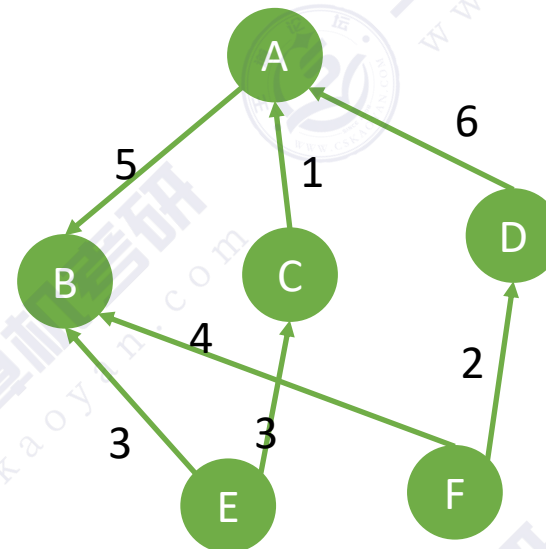
# 邻接矩阵法存储带权图（网）

无向网



|   | A        | B        | C        | D        | E        | F        |
|---|----------|----------|----------|----------|----------|----------|
| A | $\infty$ | 5        | 1        | 6        | $\infty$ | $\infty$ |
| B | 5        | $\infty$ | $\infty$ | $\infty$ | 3        | 4        |
| C | 1        | $\infty$ | $\infty$ | $\infty$ | 3        | $\infty$ |
| D | 6        | $\infty$ | $\infty$ | $\infty$ | $\infty$ | 2        |
| E | $\infty$ | 3        | 3        | $\infty$ | $\infty$ | $\infty$ |
| F | $\infty$ | 4        | $\infty$ | 2        | $\infty$ | $\infty$ |

有向网



|   | A        | B        | C        | D        | E        | F        |
|---|----------|----------|----------|----------|----------|----------|
| A | $\infty$ | 5        | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| B | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| C | 1        | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| D | 6        | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| E | $\infty$ | 3        | 3        | $\infty$ | $\infty$ | $\infty$ |
| F | $\infty$ | 4        | $\infty$ | 2        | $\infty$ | $\infty$ |

可用int的上限值表示“无穷”

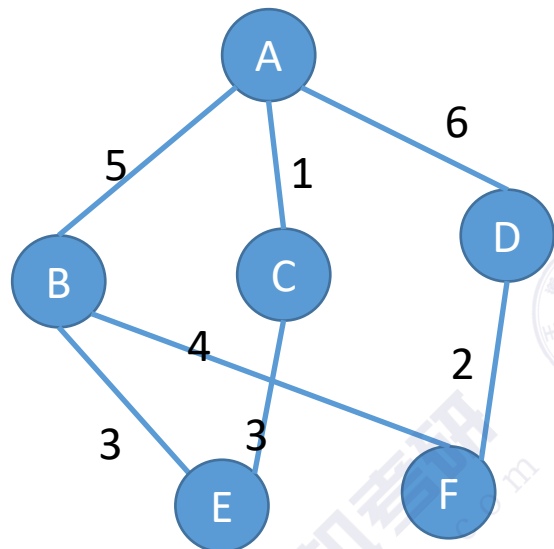
```
#define MaxVertexNum 100
#define INFINITY 最大的int值
typedef char VertexType;
typedef int EdgeType;
typedef struct{
    VertexType Vex[MaxVertexNum];
    EdgeType Edge[MaxVertexNum][MaxVertexNum];
    int vexnum, arcnum;
}MGraph;
```

```
//顶点数目的最大值
//宏定义常量"无穷"
//顶点的数据类型
//带权图中边上权值的数据类型
//顶点
//边的权
//图的当前顶点数和弧数
```



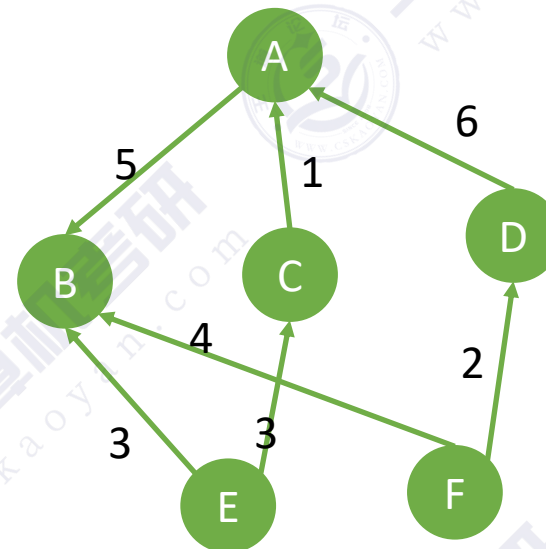
# 邻接矩阵法存储带权图（网）

无向网



|   | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A | 0 | 5 | 1 | 6 | ∞ | ∞ |
| B | 5 | 0 | ∞ | ∞ | 3 | 4 |
| C | 1 | ∞ | 0 | ∞ | 3 | ∞ |
| D | 6 | ∞ | ∞ | 0 | ∞ | 2 |
| E | ∞ | 3 | 3 | ∞ | 0 | ∞ |
| F | ∞ | 4 | ∞ | 2 | ∞ | 0 |

有向网



|   | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A | 0 | 5 | ∞ | ∞ | ∞ | ∞ |
| B | ∞ | 0 | ∞ | ∞ | ∞ | ∞ |
| C | 1 | ∞ | 0 | ∞ | ∞ | ∞ |
| D | 6 | ∞ | ∞ | 0 | ∞ | ∞ |
| E | ∞ | 3 | 3 | ∞ | 0 | ∞ |
| F | ∞ | 4 | ∞ | 2 | ∞ | 0 |

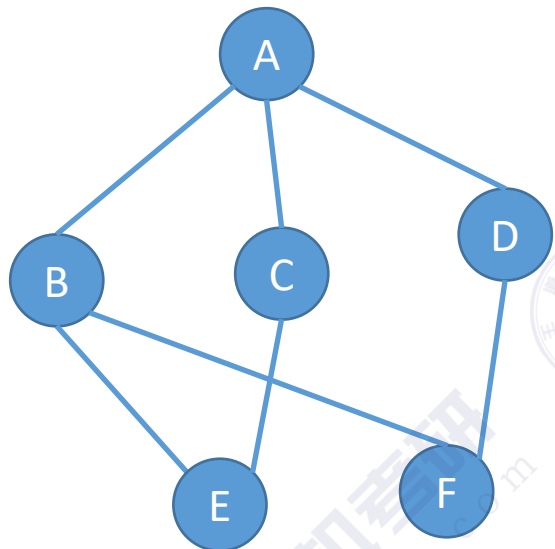
可用int的上限值表示“无穷”

```
#define MaxVertexNum 100
#define INFINITY 最大的int值
typedef char VertexType;
typedef int EdgeType;
typedef struct{
    VertexType Vex[MaxVertexNum];
    EdgeType Edge[MaxVertexNum][MaxVertexNum];
    int vexnum, arcnum;
}MGraph;
```

//顶点数目的最大值  
 //宏定义常量“无穷”  
 //顶点的数据类型  
 //带权图中边上权值的数据类型  
 //顶点  
 //边的权  
 //图的当前顶点数和弧数

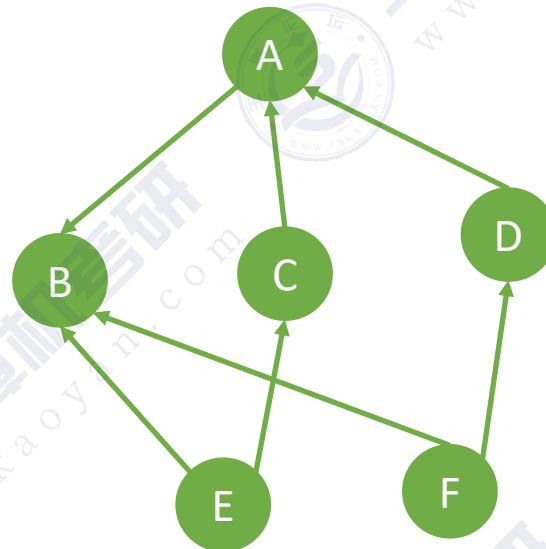
## 邻接矩阵法的性能分析

无向图



|   | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A | 0 | 1 | 1 | 1 | 0 | 0 |
| B | 1 | 0 | 0 | 0 | 1 | 1 |
| C | 1 | 0 | 0 | 0 | 1 | 0 |
| D | 1 | 0 | 0 | 0 | 0 | 1 |
| E | 0 | 1 | 1 | 0 | 0 | 0 |
| F | 0 | 1 | 0 | 1 | 0 | 0 |

有向图



|   | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A | 0 | 1 | 0 | 0 | 0 | 0 |
| B | 0 | 0 | 0 | 0 | 0 | 0 |
| C | 1 | 0 | 0 | 0 | 0 | 0 |
| D | 1 | 0 | 0 | 0 | 0 | 0 |
| E | 0 | 1 | 1 | 0 | 0 | 0 |
| F | 0 | 1 | 0 | 1 | 0 | 0 |

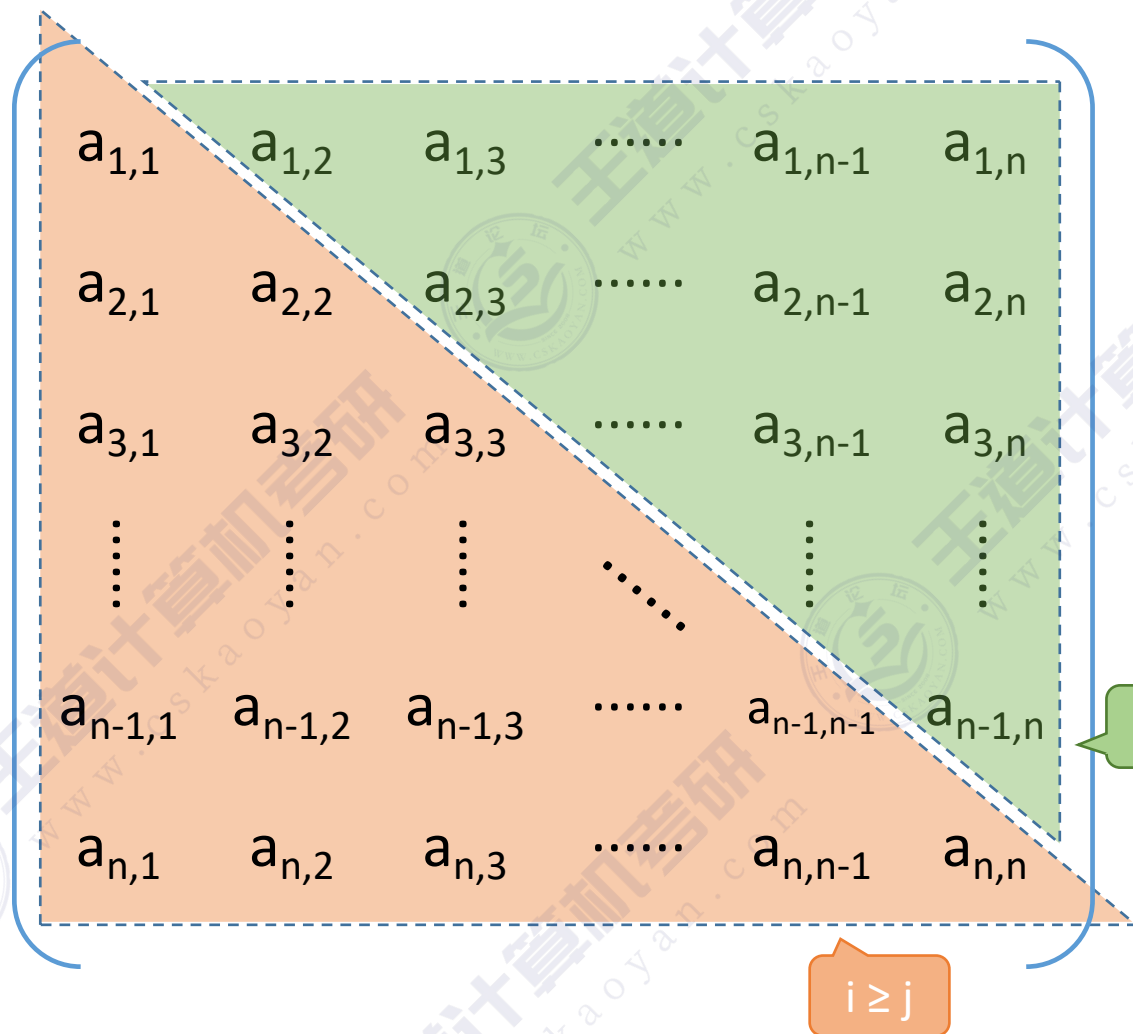
空间复杂度:  $O(|V|^2)$  ——只和顶点数相关, 和实际的边数无关

适合用于存储稠密图

无向图的邻接矩阵是对称矩阵, 可以压缩存储 (只存储上三角区/下三角区)



## 回顾：对称矩阵的压缩存储



策略：只存储主对角线+下三角区

按行优先原则将各元素存入一维数组中。

| B[0]             | B[1]             | B[2]             | B[3]             | ....  | B[ $\frac{n(n+1)}{2}$ -1] |                  |
|------------------|------------------|------------------|------------------|-------|---------------------------|------------------|
| a <sub>1,1</sub> | a <sub>2,1</sub> | a <sub>2,2</sub> | a <sub>3,1</sub> | ..... | a <sub>n,n-1</sub>        | a <sub>n,n</sub> |

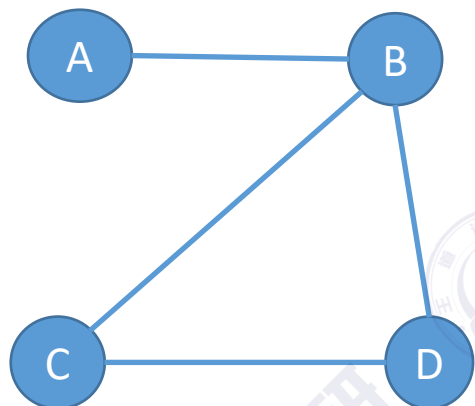
矩阵下标  $\rightarrow$  一维数组下标

$a_{i,j} \rightarrow B[k]$

$a_{i,j} = a_{j,i}$  (对称矩阵性质)

$$k = \begin{cases} \frac{i(i-1)}{2} + j - 1, & i \geq j \text{ (下三角区和主对角线元素)} \\ \frac{j(j-1)}{2} + i - 1, & i < j \text{ (上三角区元素 } a_{ij} = a_{ji} \text{)} \end{cases}$$

## 邻接矩阵法的性质



|   | A | B | C | D |
|---|---|---|---|---|
| A | 0 | 1 | 0 | 0 |
| B | 1 | 0 | 1 | 1 |
| C | 0 | 1 | 0 | 1 |
| D | 0 | 1 | 1 | 0 |

设图 $G$ 的邻接矩阵为 $A$ （矩阵元素为0/1），则 $A^n$ 的元素 $A^n[i][j]$ 等于由顶点 $i$ 到顶点 $j$ 的长度为 $n$ 的路径的数目

|   |   |   |   |
|---|---|---|---|
| 0 | 1 | 0 | 0 |
| 1 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 |
| 0 | 1 | 1 | 0 |

\*

|   |   |   |   |
|---|---|---|---|
| 0 | 1 | 0 | 0 |
| 1 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 |
| 0 | 1 | 1 | 0 |

$$A^2[1][4] = a_{1,1} a_{1,4} + a_{1,2} a_{2,4} + a_{1,3} a_{3,4} + a_{1,4} a_{4,4} = 1$$

$$A^2[2][2] = a_{2,1} a_{1,2} + a_{2,2} a_{2,2} + a_{2,3} a_{3,2} + a_{2,4} a_{4,2} = 3$$

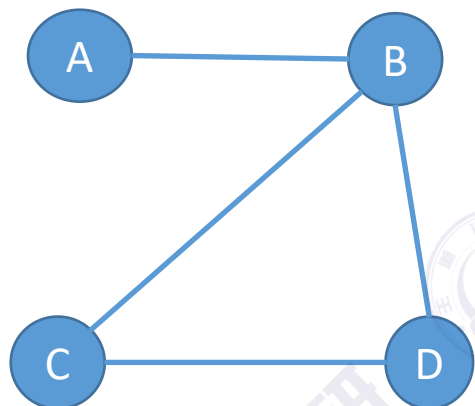
$$A^2[3][3] = a_{3,1} a_{1,3} + a_{3,2} a_{2,3} + a_{3,3} a_{3,3} + a_{3,4} a_{4,3} = 1$$

$$A^2[1][2] = a_{1,1} a_{1,2} + a_{1,2} a_{2,2} + a_{1,3} a_{3,2} + a_{1,4} a_{4,2} = 1$$

$$A^2 =$$

|   |   |   |   |
|---|---|---|---|
| 1 | 0 | 1 | 1 |
| 0 | 3 | 1 | 1 |
| 1 | 1 | 2 | 1 |
| 1 | 1 | 1 | 2 |

## 邻接矩阵法的性质



|   | A | B | C | D |
|---|---|---|---|---|
| A | 0 | 1 | 0 | 0 |
| B | 1 | 0 | 1 | 1 |
| C | 0 | 1 | 0 | 1 |
| D | 0 | 1 | 1 | 0 |

设图 $G$ 的邻接矩阵为 $A$ （矩阵元素为0/1），则 $A^n$ 的元素 $A^n[i][j]$ 等于由顶点 $i$ 到顶点 $j$ 的长度为 $n$ 的路径的数目

$$A^3 = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 3 & 1 & 1 \\ 1 & 1 & 2 & 1 \\ 1 & 1 & 1 & 2 \end{bmatrix} * \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 3 & 1 & 1 \\ 3 & 2 & 4 & 4 \\ 1 & 4 & 2 & 3 \\ 1 & 4 & 3 & 2 \end{bmatrix}$$

## 知识回顾与重要考点

邻接矩阵法要点回顾:

- 如何计算指定顶点的度、入度、出度（分无向图、有向图来考虑）？时间复杂度如何？
- 如何找到与顶点相邻的边（入边、出边）？时间复杂度如何？
- 如何存储带权图？
- 空间复杂度—— $O(|V|^2)$ ，适合存储稠密图
- 无向图的邻接矩阵为对称矩阵，如何压缩存储？
- 设图G的邻接矩阵为A（矩阵元素为0/1），则 $A^n$ 的元素 $A^n[i][j]$ 等于由顶点i到顶点j的长度为n的路径的数目

本节内容

图的存储

邻接表法

## 知识总览

### 图的存储

邻接矩阵

数组实现的顺序存储，  
空间复杂度高，不适合  
存储稀疏图

邻接表

顺序+链式存储

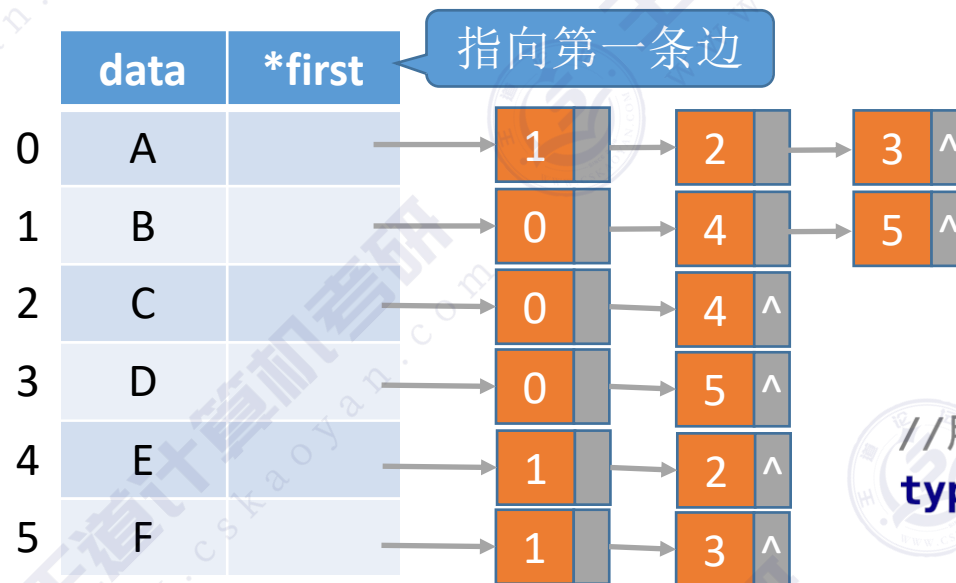
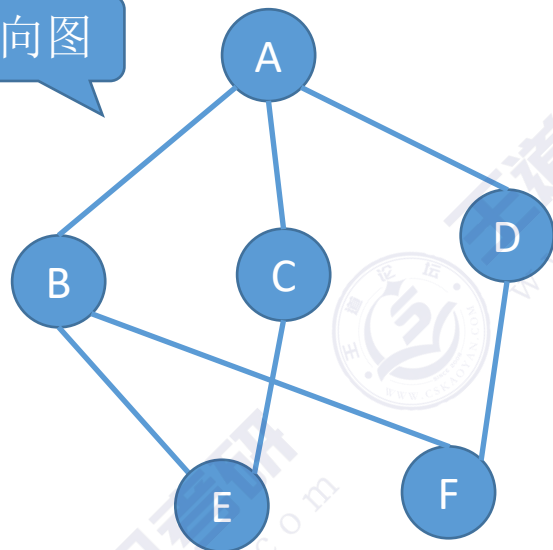
十字链表

邻接多重表



## 邻接表法（顺序+链式存储）

无向图



//用邻接表存储的图

```
typedef struct{
    AdjList vertices;
    int vexnum, arcnum;
} ALGraph;
```

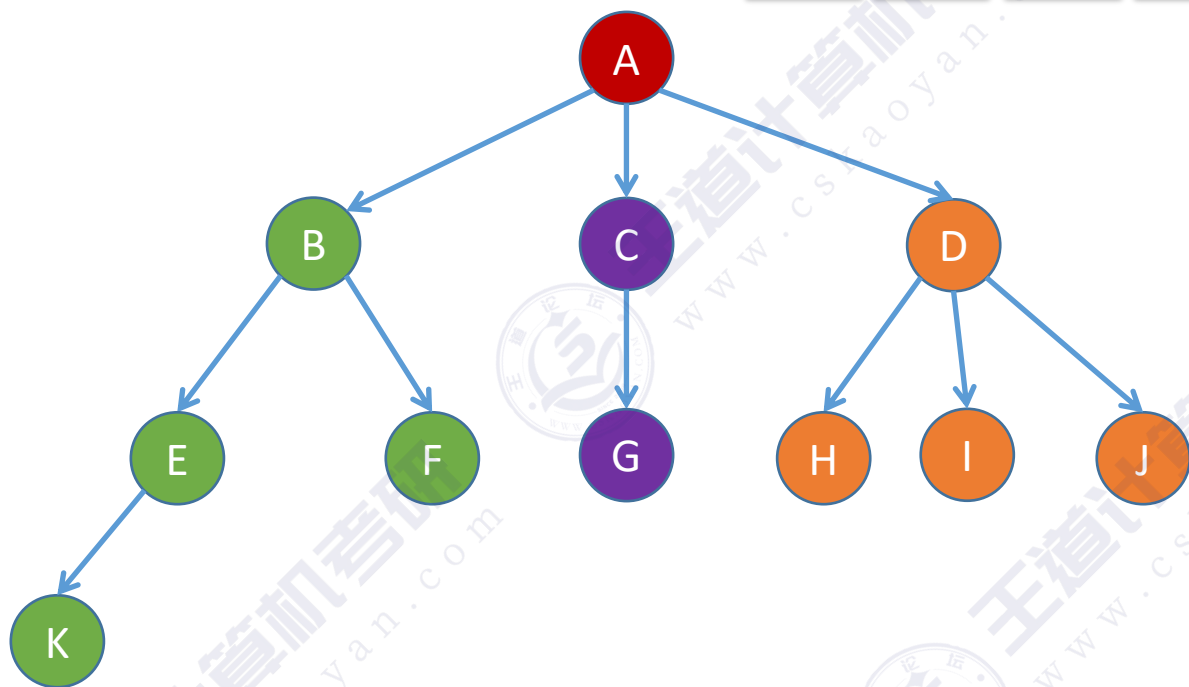
// "边/弧"

```
typedef struct ArcNode{
    int adjvex; //边/弧指向哪个结点
    struct ArcNode *next; //指向下一条弧的指针
    //InfoType info; //边权值
} ArcNode;
```

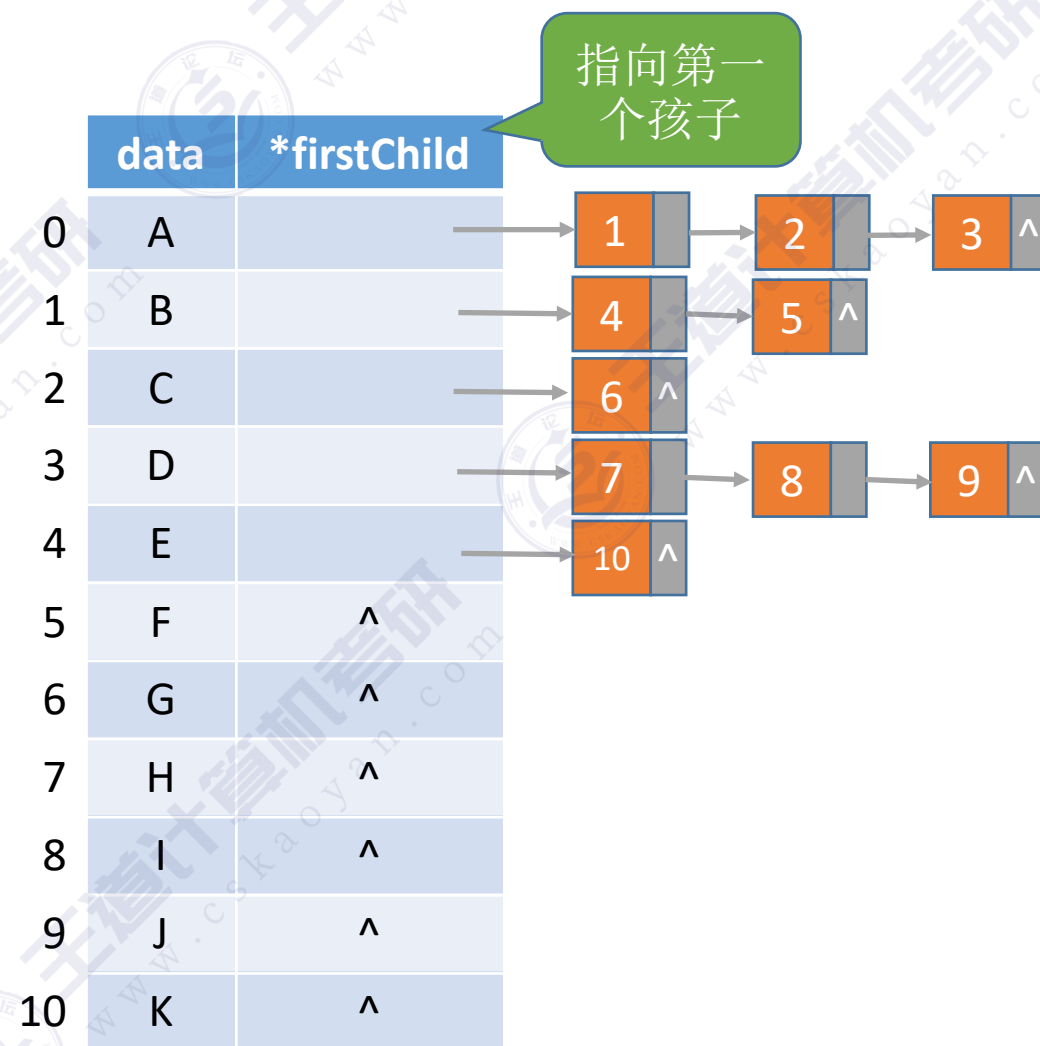
// "顶点"

```
typedef struct VNode{
    VertexType data; //顶点信息
    ArcNode *first; //第一条边/弧
} VNode, AdjList[MaxVertexNum];
```

## 对比：树的孩子表示法

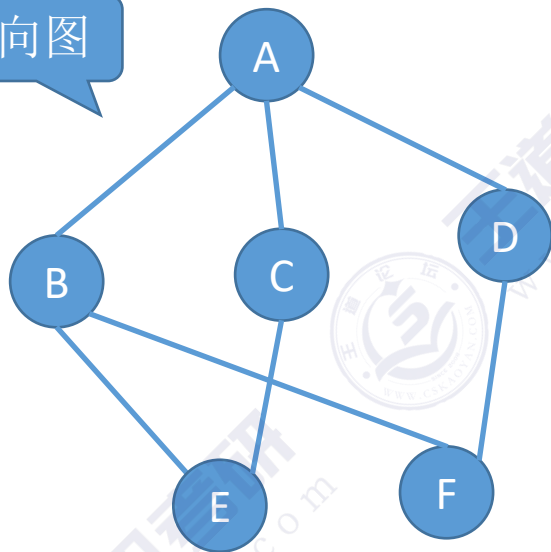


孩子表示法：顺序存储各个节点，每个结点中保存孩子链表头指针



## 邻接表法

无向图



|   | data | *first |
|---|------|--------|
| 0 | A    |        |
| 1 | B    |        |
| 2 | C    |        |
| 3 | D    |        |
| 4 | E    |        |
| 5 | F    |        |

|   |   |   |   |
|---|---|---|---|
| 1 | 2 | 3 | ^ |
| 0 | 4 | 5 | ^ |
| 0 | 4 | ^ |   |
| 0 | 5 | ^ |   |
| 1 | 2 | ^ |   |
| 1 | 3 | ^ |   |

边结点的数量是 $2|E|$ ，  
整体空间复杂度为  
 $O(|V| + 2|E|)$

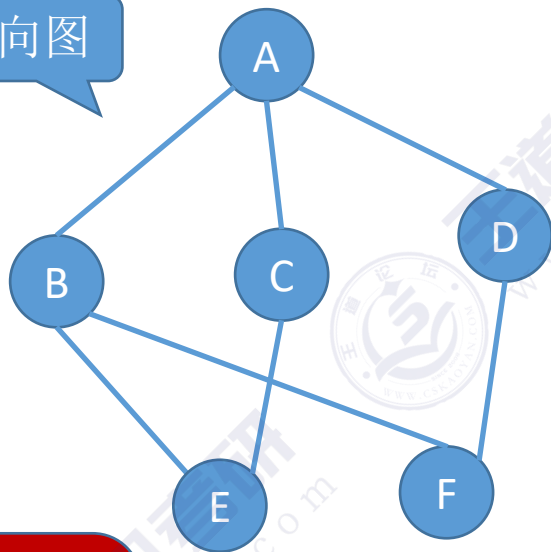
思考：如何求顶点的  
度、入度、出度？

如何找到与一个顶  
点相连的边/弧？

边结点的数量是 $|E|$ ，  
整体空间复杂度为  
 $O(|V| + |E|)$

## 邻接表法

无向图



只要确定了顶点编号，图的邻接矩阵表示方式唯一

|   | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A | 0 | 1 | 1 | 1 | 0 | 0 |
| B | 1 | 0 | 0 | 0 | 1 | 1 |
| C | 1 | 0 | 0 | 0 | 1 | 0 |
| D | 1 | 0 | 0 | 0 | 0 | 1 |
| E | 0 | 1 | 1 | 0 | 0 | 0 |
| F | 0 | 1 | 0 | 1 | 0 | 0 |

|   | data | *first |
|---|------|--------|
| 0 | A    |        |
| 1 | B    |        |
| 2 | C    |        |
| 3 | D    |        |
| 4 | E    |        |
| 5 | F    |        |

Adjacency list representation 1:

- 0 (A) → 1 (B) → 2 (C) → 3 (D) → ^
- 1 (B) → 0 (A) → 4 (E) → 5 (F) → ^
- 2 (C) → 0 (A) → 4 (E) → ^
- 3 (D) → 0 (A) → 5 (F) → ^
- 4 (E) → 1 (B) → 2 (C) → ^
- 5 (F) → 1 (B) → 3 (D) → ^

图的邻接表表示方式并不唯一

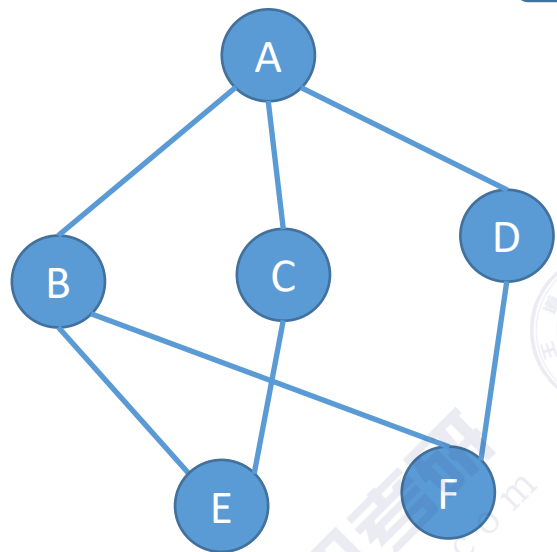
|   | data | *first |
|---|------|--------|
| 0 | A    |        |
| 1 | B    |        |
| 2 | C    |        |
| 3 | D    |        |
| 4 | E    |        |
| 5 | F    |        |

Adjacency list representation 2:

- 0 (A) → 3 (D) → 2 (C) → 1 (B) → ^
- 1 (B) → 0 (A) → 4 (E) → 5 (F) → ^
- 2 (C) → 4 (E) → 0 (A) → ^
- 3 (D) → 0 (A) → 5 (F) → ^
- 4 (E) → 1 (B) → 2 (C) → ^
- 5 (F) → 1 (B) → 3 (D) → ^

## 知识回顾与重要考点

邻接表



|   | data | *first        |
|---|------|---------------|
| 0 | A    | → 1 → 2 → 3 ^ |
| 1 | B    | → 0 → 4 → 5 ^ |
| 2 | C    | → 0 → 4 ^     |
| 3 | D    | → 0 → 5 ^     |
| 4 | E    | → 1 → 2 ^     |
| 5 | F    | → 1 → 3 ^     |

邻接矩阵

|   | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A | 0 | 1 | 1 | 1 | 0 | 0 |
| B | 1 | 0 | 0 | 0 | 1 | 1 |
| C | 1 | 0 | 0 | 0 | 1 | 0 |
| D | 1 | 0 | 0 | 0 | 0 | 1 |
| E | 0 | 1 | 1 | 0 | 0 | 0 |
| F | 0 | 1 | 0 | 1 | 0 | 0 |

|           | 邻接表                                     | 邻接矩阵       |
|-----------|-----------------------------------------|------------|
| 空间复杂度     | 无向图 $O( V  + 2 E )$ ；有向图 $O( V  +  E )$ | $O( V ^2)$ |
| 适合用于      | 存储稀疏图                                   | 存储稠密图      |
| 表示方式      | 不唯一                                     | 唯一         |
| 计算度/出度/入度 | 计算有向图的度、入度不方便，其余很方便                     | 必须遍历对应行或列  |
| 找相邻的边     | 找有向图的入边不方便，其余很方便                        | 必须遍历对应行或列  |

本节内容

# 图的存储

十字链表、  
邻接多重表

# 知识总览

## 图的存储

邻接矩阵

邻接表

十字链表

邻接多重表

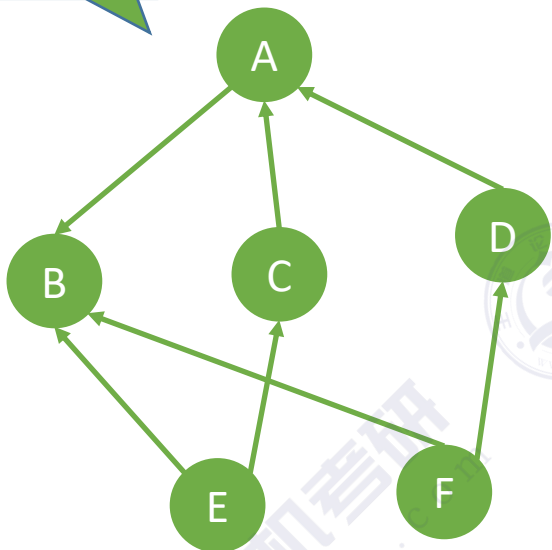
存储有向图

存储无向图



# 邻接矩阵、邻接表存储有向图

有向图



|   | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A | 0 | 1 | 1 | 1 | 0 | 0 |
| B | 1 | 0 | 0 | 0 | 1 | 1 |
| C | 1 | 0 | 0 | 0 | 1 | 0 |
| D | 1 | 0 | 0 | 0 | 0 | 1 |
| E | 0 | 1 | 1 | 0 | 0 | 0 |
| F | 0 | 1 | 0 | 1 | 0 | 0 |

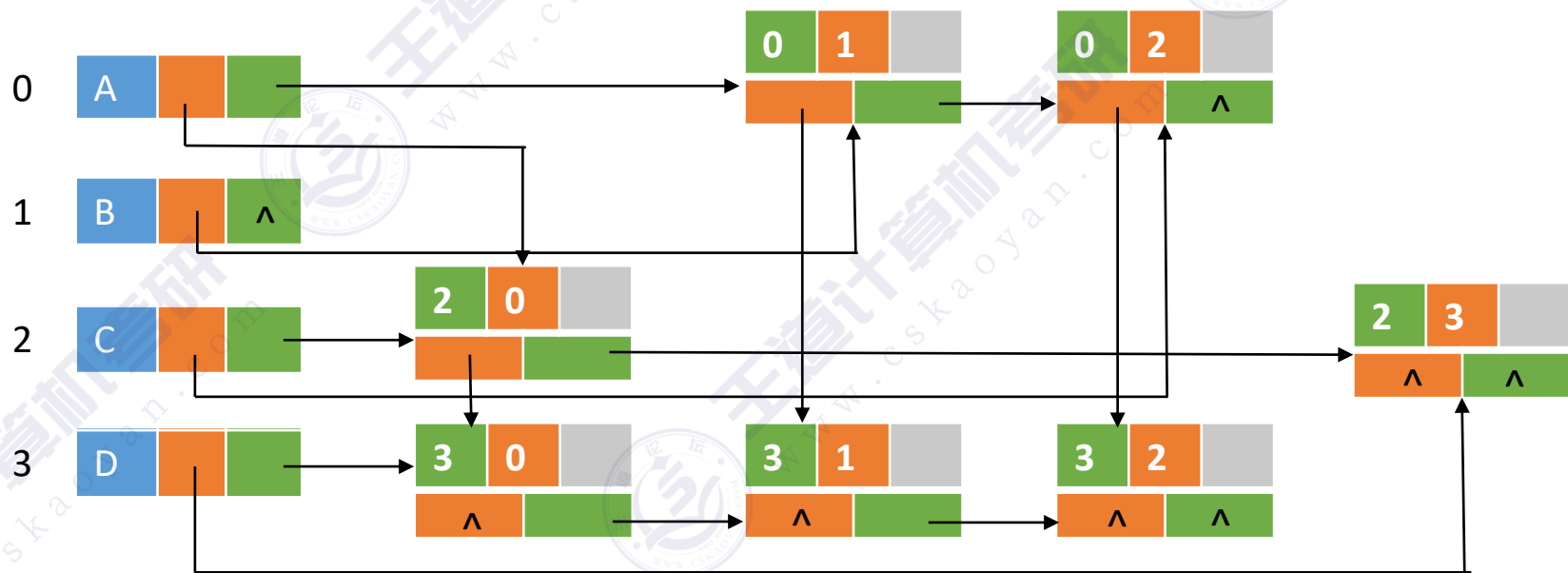
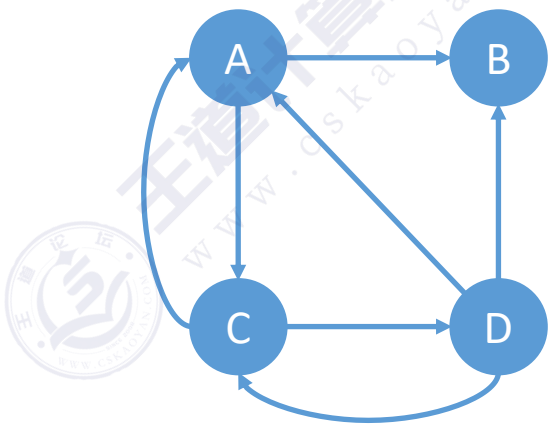
|   | data | *first |
|---|------|--------|
| 0 | A    | 1 ^    |
| 1 | B    | ^      |
| 2 | C    | 0 ^    |
| 3 | D    | 0 ^    |
| 4 | E    | 1 2 ^  |
| 5 | F    | 1 3 ^  |

找顶点的入边不方便

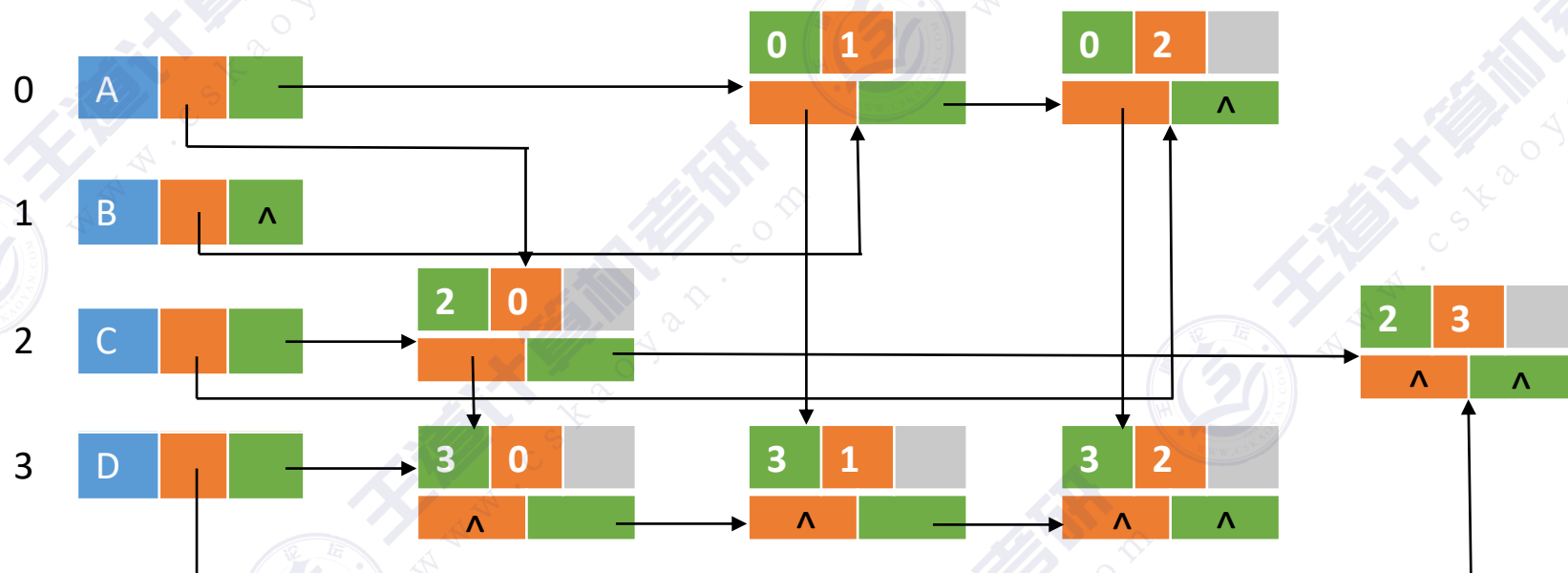
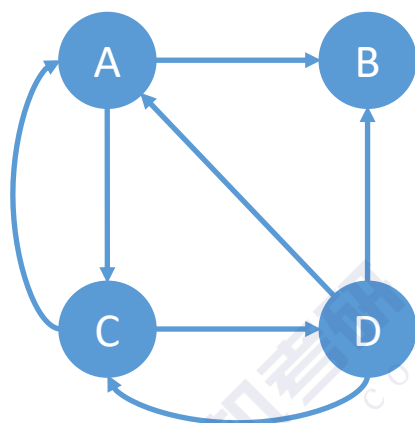
|           | 邻接表                                     | 邻接矩阵       |
|-----------|-----------------------------------------|------------|
| 空间复杂度     | 无向图 $O( V  + 2 E )$ ；有向图 $O( V  +  E )$ | $O( V ^2)$ |
| 计算度/出度/入度 | 计算有向图的度、入度不方便，其余很方便                     | 必须遍历对应行或列  |
| 找相邻的边     | 找有向图的入边不方便，其余很方便                        | 必须遍历对应行或列  |

空间复杂度  
高  $O(|V|^2)$

# 十字链表存储有向图



## 十字链表法性能分析



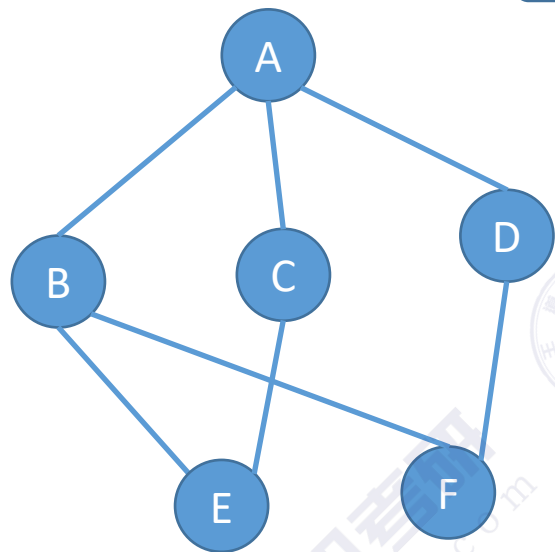
空间复杂度:  $O(|V|+|E|)$

如何找到指定顶点的所有出边? ——顺着绿色线路找  
如何找到指定顶点的所有入边? ——顺着橙色线路找

注意: 十字链表只用于存储有向图

# 邻接矩阵、邻接表存储无向图

邻接表



|   | data | *first |
|---|------|--------|
| 0 | A    |        |
| 1 | B    |        |
| 2 | C    |        |
| 3 | D    |        |
| 4 | E    |        |
| 5 | F    |        |

Adjacency list representation showing the mapping from vertex indices to their neighbors:

- 0 (A) points to 1 (B), 2 (C), 3 (D)
- 1 (B) points to 0 (A), 4 (E)
- 2 (C) points to 0 (A), 4 (E)
- 3 (D) points to 0 (A), 5 (F)
- 4 (E) points to 1 (B), 2 (C)
- 5 (F) points to 3 (D)

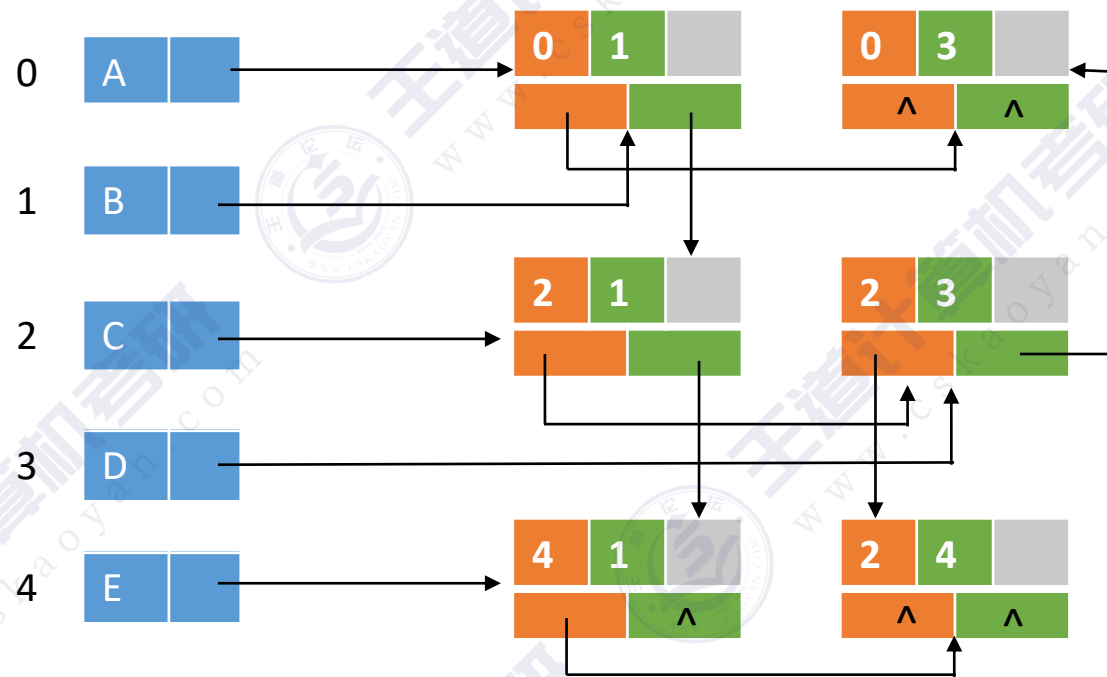
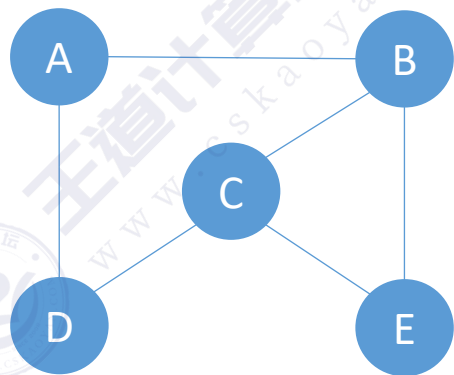
邻接矩阵

|   | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A | 0 | 1 | 1 | 1 | 0 | 0 |
| B | 1 | 0 | 0 | 0 | 1 | 1 |
| C | 1 | 0 | 0 | 0 | 1 | 0 |
| D | 1 | 0 | 0 | 0 | 0 | 1 |
| E | 0 | 1 | 1 | 0 | 0 | 0 |
| F | 0 | 1 | 0 | 1 | 0 | 0 |

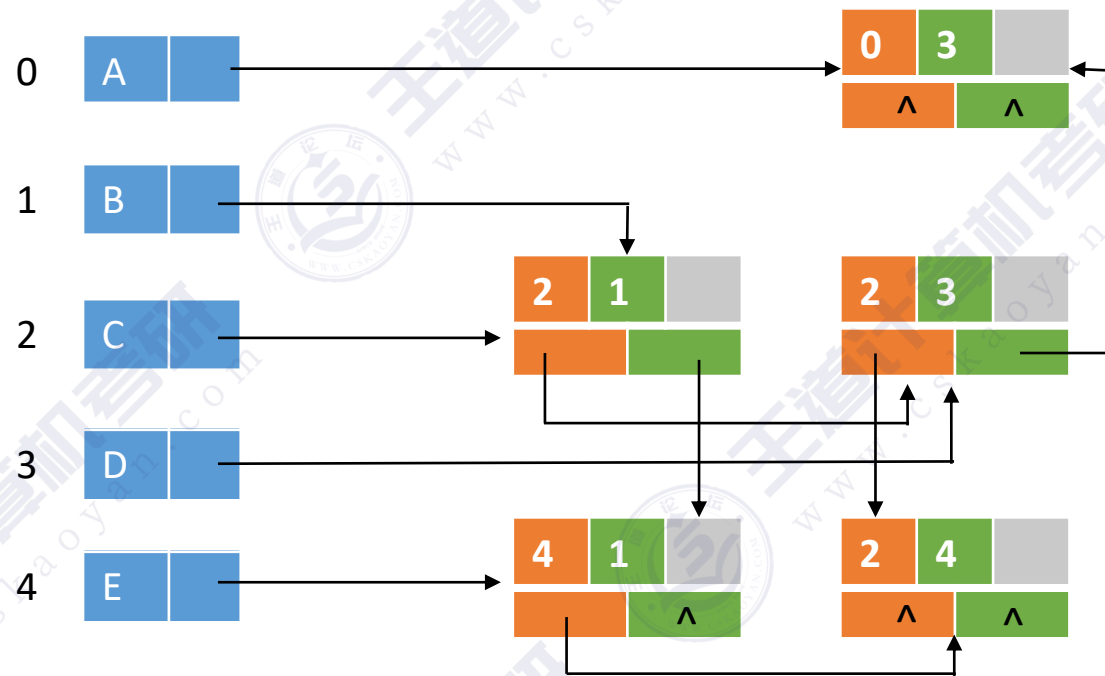
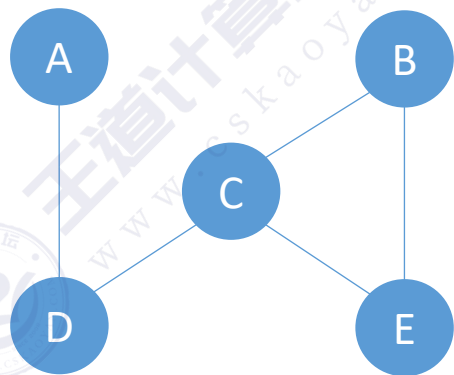
每条边对应两份冗余信息，  
删除顶点、删除边等操作  
时间复杂度高

空间复杂度高  
 $O(|V|^2)$

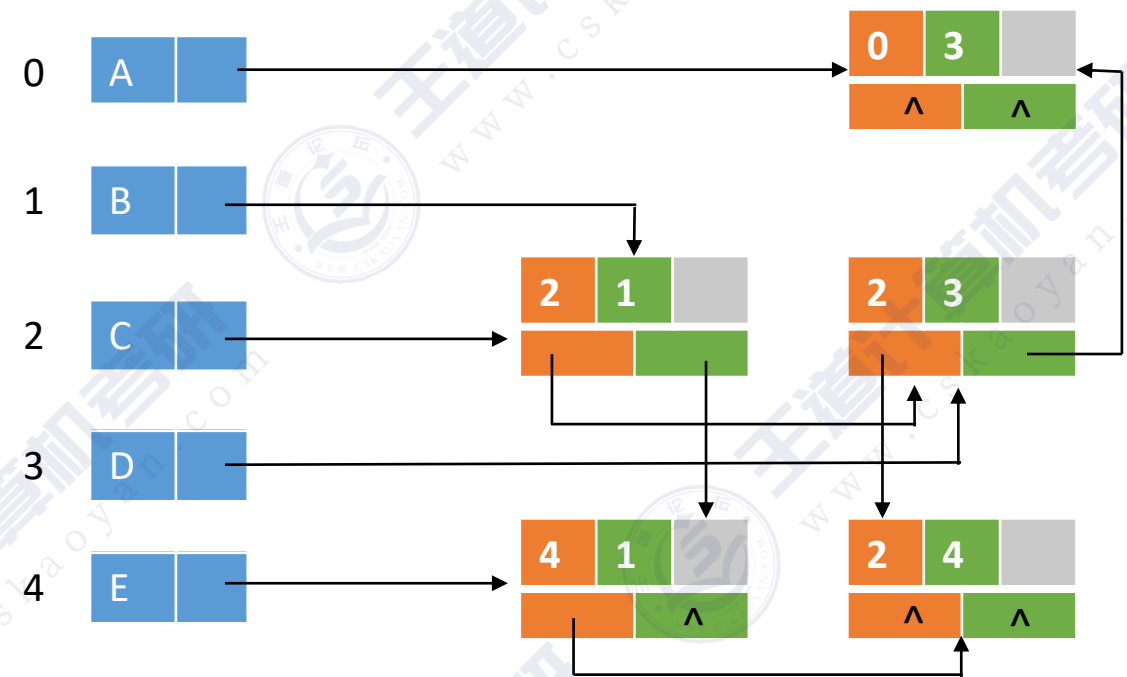
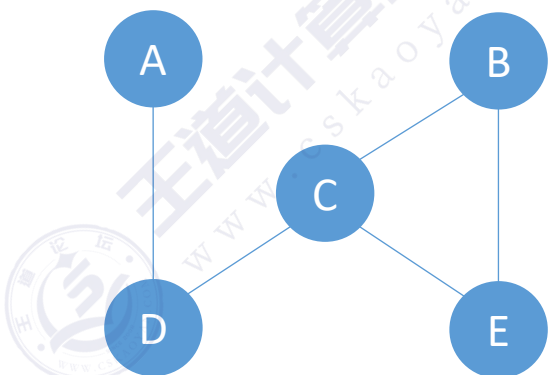
# 邻接多重表存储无向图



# 邻接多重表存储无向图

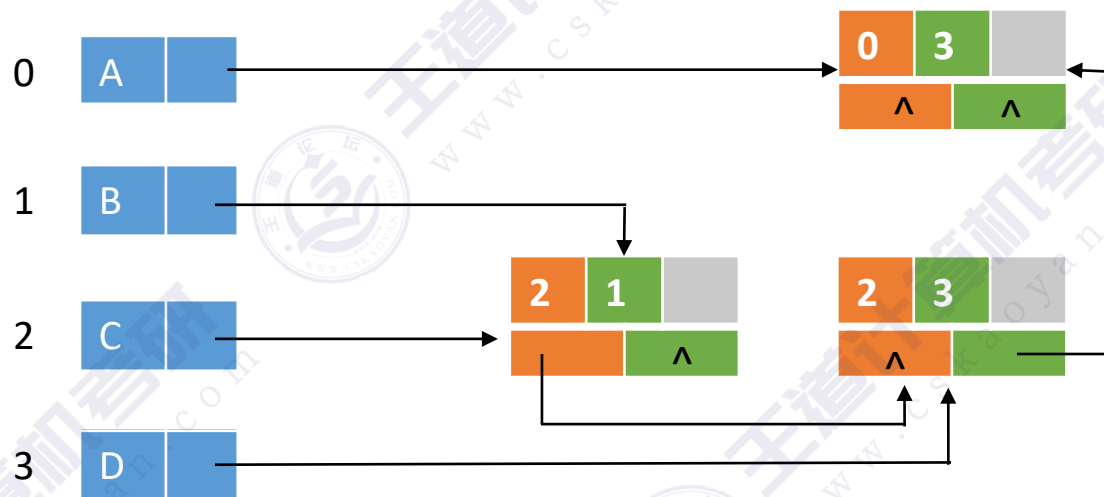
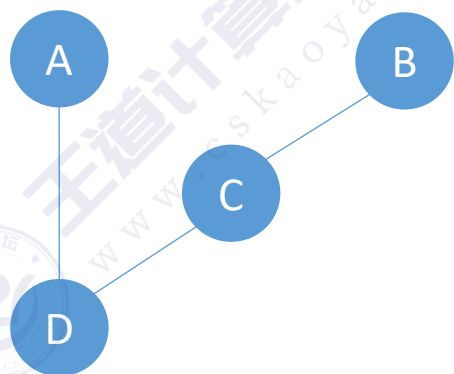


# 邻接多重表存储无向图





# 邻接多重表存储无向图



每条边只对应一份数据

空间复杂度:  $O(|V|+|E|)$

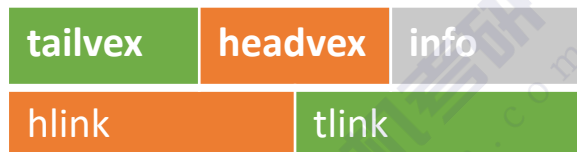
删除边、删除节点等操作很方便

注意: 邻接多重表只适用于存储无向图

## 知识回顾与重要考点

|        | 邻接矩阵                       | 邻接表                                       | 十字链表           | 邻接多重表          |
|--------|----------------------------|-------------------------------------------|----------------|----------------|
| 空间复杂度  | $O( V ^2)$                 | 无向图 $O( V  + 2 E )$<br>有向图 $O( V  +  E )$ | $O( V  +  E )$ | $O( V  +  E )$ |
| 找相邻边   | 遍历对应行或列<br>时间复杂度为 $O( V )$ | 找有向图的入边必须遍历整个邻接表                          | 很方便            | 很方便            |
| 删除边或顶点 | 删除边很方便，删除顶点需要大量移动数据        | 无向图中删除边或顶点都不方便                            | 很方便            | 很方便            |
| 适用于    | 稠密图                        | 稀疏图和其他                                    | 只能存有向图         | 只能存无向图         |
| 表示方式   | 唯一                         | 不唯一                                       | 不唯一            | 不唯一            |

弧结点:



顶点结点:



十字链表

边结点:



顶点结点:



邻接多重表

本节内容

图

基本操作

# 知识总览

图的基本操作:

- $\text{Adjacent}(G, x, y)$ : 判断图 $G$ 是否存在边 $\langle x, y \rangle$ 或 $(x, y)$ 。
- $\text{Neighbors}(G, x)$ : 列出图 $G$ 中与结点 $x$ 邻接的边。
- $\text{InsertVertex}(G, x)$ : 在图 $G$ 中插入顶点 $x$ 。
- $\text{DeleteVertex}(G, x)$ : 从图 $G$ 中删除顶点 $x$ 。
- $\text{AddEdge}(G, x, y)$ : 若无向边 $(x, y)$ 或有向边 $\langle x, y \rangle$ 不存在, 则向图 $G$ 中添加该边。
- $\text{RemoveEdge}(G, x, y)$ : 若无向边 $(x, y)$ 或有向边 $\langle x, y \rangle$ 存在, 则从图 $G$ 中删除该边。
- $\text{FirstNeighbor}(G, x)$ : 求图 $G$ 中顶点 $x$ 的第一个邻接点, 若有则返回顶点号。若 $x$ 没有邻接点或图中不存在 $x$ , 则返回-1。
- $\text{NextNeighbor}(G, x, y)$ : 假设图 $G$ 中顶点 $y$ 是顶点 $x$ 的一个邻接点, 返回除 $y$ 之外顶点 $x$ 的下一个邻接点的顶点号, 若 $y$ 是 $x$ 的最后一个邻接点, 则返回-1。
- $\text{Get\_edge\_value}(G, x, y)$ : 获取图 $G$ 中边 $(x, y)$ 或 $\langle x, y \rangle$ 对应的权值。
- $\text{Set\_edge\_value}(G, x, y, v)$ : 设置图 $G$ 中边 $(x, y)$ 或 $\langle x, y \rangle$ 对应的权值为 $v$ 。

图的存储

邻接矩阵

邻接表

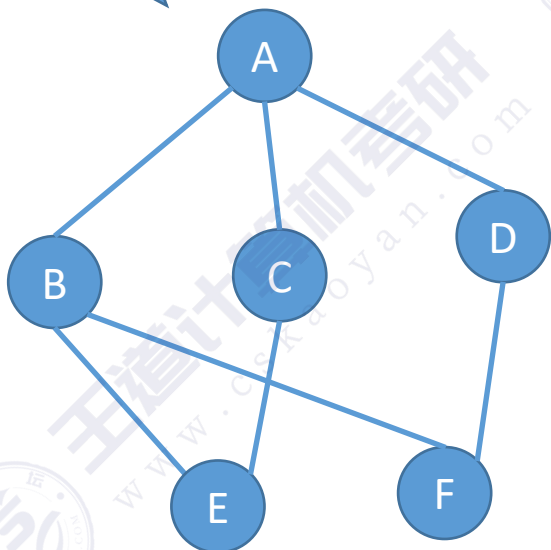
十字链表

邻接多重表

# 图的基本操作

- Adjacent(G,x,y): 判断图G是否存在边<x, y>或(x, y)。

无向图



邻接矩阵

$O(1)$

|   | data |   | A | B | C | D | E | F |
|---|------|---|---|---|---|---|---|---|
| 0 | A    | A | 0 | 1 | 1 | 1 | 0 | 0 |
| 1 | B    | B | 1 | 0 | 0 | 0 | 1 | 1 |
| 2 | C    | C | 1 | 0 | 0 | 0 | 1 | 0 |
| 3 | D    | D | 1 | 0 | 0 | 0 | 0 | 1 |
| 4 | E    | E | 0 | 1 | 1 | 0 | 0 | 0 |
| 5 | F    | F | 0 | 1 | 0 | 1 | 0 | 0 |

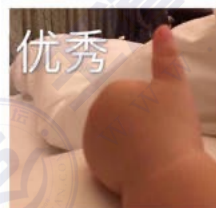
邻接表

$O(1) \sim O(|V|)$

|   | data | *first              |
|---|------|---------------------|
| 0 | A    | → [1] → [2] → [3] ^ |
| 1 | B    | → [0] → [4] → [5] ^ |
| 2 | C    | → [0] → [4] ^       |
| 3 | D    | → [0] → [5] ^       |
| 4 | E    | → [1] → [2] ^       |
| 5 | F    | → [1] → [3] ^       |

# 图的基本操作

- Adjacent(G,x,y): 判断图G是否存在边<x, y>或(x, y)。



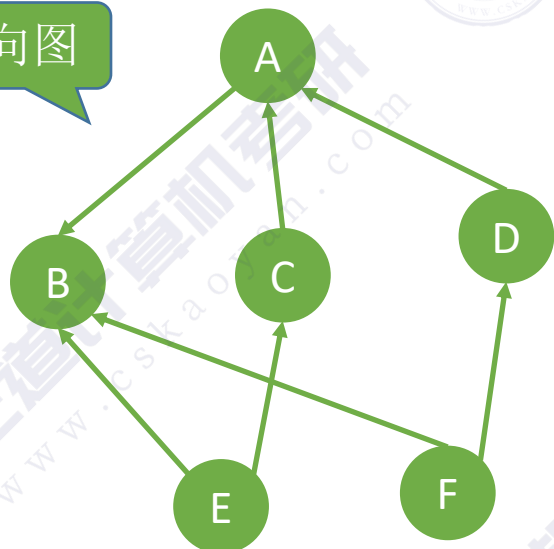
邻接矩阵

$O(1)$

邻接表

$O(1) \sim O(|V|)$

有向图



|   | data |   | A | B | C | D | E | F |
|---|------|---|---|---|---|---|---|---|
| 0 | A    | A | 0 | 1 | 0 | 0 | 0 | 0 |
| 1 | B    | B | 0 | 0 | 0 | 0 | 0 | 0 |
| 2 | C    | C | 1 | 0 | 0 | 0 | 0 | 0 |
| 3 | D    | D | 1 | 0 | 0 | 0 | 0 | 0 |
| 4 | E    | E | 0 | 1 | 1 | 0 | 0 | 0 |
| 5 | F    | F | 0 | 1 | 0 | 1 | 0 | 0 |

|   | data | *first    |
|---|------|-----------|
| 0 | A    | → 1 ^     |
| 1 | B    | → ^       |
| 2 | C    | → 0 ^     |
| 3 | D    | → 0 ^     |
| 4 | E    | → 1 → 2 ^ |
| 5 | F    | → 1 → 3 ^ |

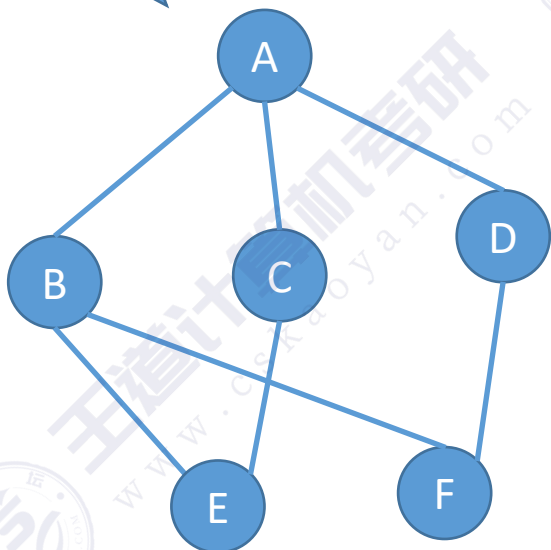


# 图的基本操作

- Neighbors(G,x): 列出图G中与结点x邻接的边。



无向图



邻接矩阵

$O(|V|)$

|   | data |   | A | B | C | D | E | F |
|---|------|---|---|---|---|---|---|---|
| 0 | A    | A | 0 | 1 | 1 | 1 | 0 | 0 |
| 1 | B    | B | 1 | 0 | 0 | 0 | 1 | 1 |
| 2 | C    | C | 1 | 0 | 0 | 0 | 1 | 0 |
| 3 | D    | D | 1 | 0 | 0 | 0 | 0 | 1 |
| 4 | E    | E | 0 | 1 | 1 | 0 | 0 | 0 |
| 5 | F    | F | 0 | 1 | 0 | 1 | 0 | 0 |

邻接表

$O(1) \sim O(|V|)$

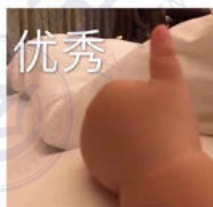
|   | data | *first              |
|---|------|---------------------|
| 0 | A    | → [1] → [2] → [3] ^ |
| 1 | B    | → [0] → [4] → [5] ^ |
| 2 | C    | → [0] → [4] ^       |
| 3 | D    | → [0] → [5] ^       |
| 4 | E    | → [1] → [2] ^       |
| 5 | F    | → [1] → [3] ^       |



# 图的基本操作

- Neighbors(G,x): 列出图G中与结点x邻接的边。

万一是个稀疏图呢?



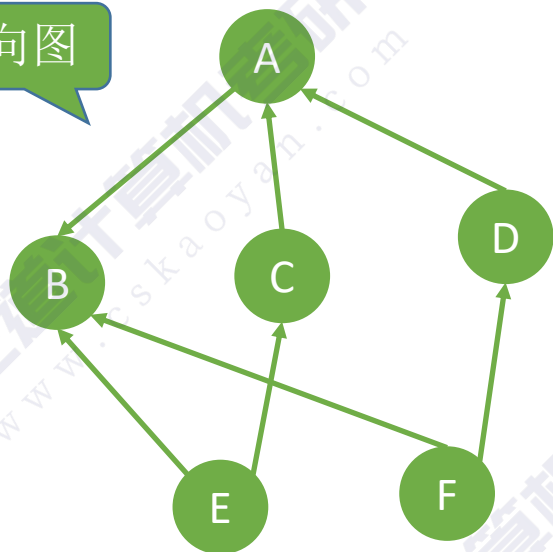
邻接矩阵

$O(|V|)$

邻接表

出边:  $O(1) \sim O(|V|)$   
入边:  $O(|E|)$

有向图



|   | data |   | A | B | C | D | E | F |
|---|------|---|---|---|---|---|---|---|
| 0 | A    | A | 0 | 1 | 0 | 0 | 0 | 0 |
| 1 | B    | B | 0 | 0 | 0 | 0 | 0 | 0 |
| 2 | C    | C | 1 | 0 | 0 | 0 | 0 | 0 |
| 3 | D    | D | 1 | 0 | 0 | 0 | 0 | 0 |
| 4 | E    | E | 0 | 1 | 1 | 0 | 0 | 0 |
| 5 | F    | F | 0 | 1 | 0 | 1 | 0 | 0 |

|   | data | *first    |
|---|------|-----------|
| 0 | A    | → 1 ^     |
| 1 | B    | → ^       |
| 2 | C    | → 0 ^     |
| 3 | D    | → 0 ^     |
| 4 | E    | → 1 → 2 ^ |
| 5 | F    | → 1 → 3 ^ |

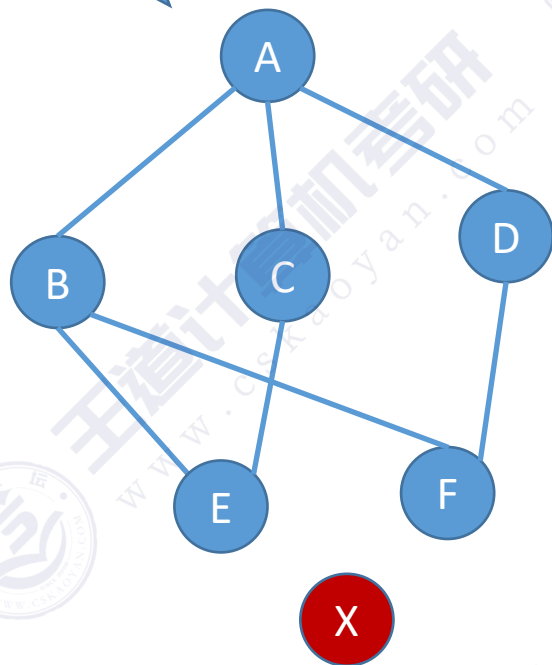
# 图的基本操作

- InsertVertex(G,x): 在图G中插入顶点x。



有向图也类似

无向图



邻接矩阵

$O(1)$

|   | data | A | B | C | D | E | F | X |
|---|------|---|---|---|---|---|---|---|
| 0 | A    | 0 | 1 | 1 | 1 | 0 | 0 | 0 |
| 1 | B    | 1 | 0 | 0 | 0 | 1 | 1 | 0 |
| 2 | C    | 1 | 0 | 0 | 0 | 1 | 0 | 0 |
| 3 | D    | 1 | 0 | 0 | 0 | 0 | 1 | 0 |
| 4 | E    | 0 | 1 | 1 | 0 | 0 | 0 | 0 |
| 5 | F    | 0 | 1 | 0 | 1 | 0 | 0 | 0 |
| 6 | X    | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

邻接表

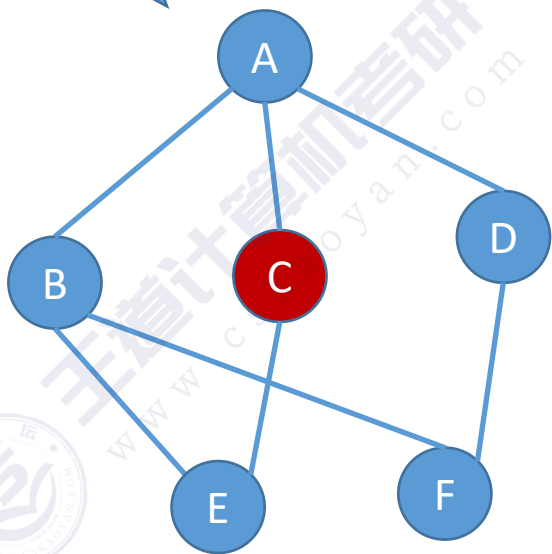
$O(1)$

|   | data | *first      |
|---|------|-------------|
| 0 | A    | 1 → 2 → 3 ^ |
| 1 | B    | 0 → 4 → 5 ^ |
| 2 | C    | 0 → 4 ^     |
| 3 | D    | 0 → 5 ^     |
| 4 | E    | 1 → 2 ^     |
| 5 | F    | 1 → 3 ^     |
| 6 | X    | ^           |

## 图的基本操作

- DeleteVertex( $G, x$ ): 从图 $G$ 中删除顶点 $x$ 。

无向图



邻接矩阵

|   | data |   | A | B | C | D | E | F |
|---|------|---|---|---|---|---|---|---|
| 0 | A    | A | 0 | 1 | 1 | 1 | 0 | 0 |
| 1 | B    | B | 1 | 0 | 0 | 0 | 1 | 1 |
| 2 | C    | C | 1 | 0 | 0 | 0 | 1 | 0 |
| 3 | D    | D | 1 | 0 | 0 | 0 | 0 | 1 |
| 4 | E    | E | 0 | 1 | 1 | 0 | 0 | 0 |
| 5 | F    | F | 0 | 1 | 0 | 1 | 0 | 0 |

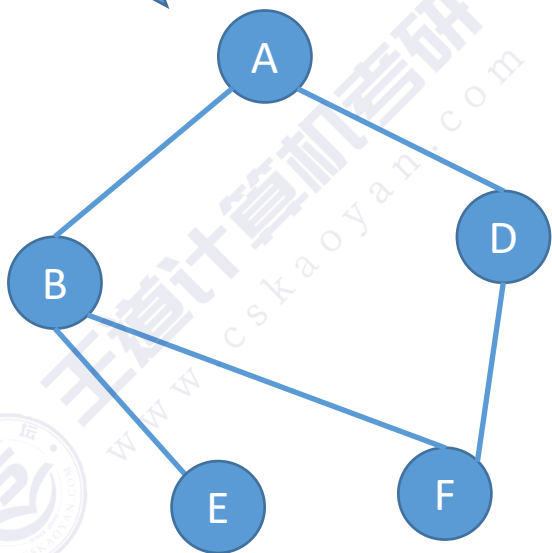
邻接表

|   | data | *first                |
|---|------|-----------------------|
| 0 | A    | → [1] → [2] → [3] → ^ |
| 1 | B    | → [0] → [4] → [5] → ^ |
| 2 | C    | → [0] → [4] → ^       |
| 3 | D    | → [0] → [5] → ^       |
| 4 | E    | → [1] → [2] → ^       |
| 5 | F    | → [1] → [3] → ^       |

## 图的基本操作

- DeleteVertex( $G, x$ ): 从图 $G$ 中删除顶点 $x$ 。

无向图



邻接矩阵

$O(|V|)$

|   | data | A | B | 空 | D | E | F |
|---|------|---|---|---|---|---|---|
| 0 | A    | 0 | 1 | 0 | 1 | 0 | 0 |
| 1 | B    | 1 | 0 | 0 | 0 | 1 | 1 |
| 2 | 空    | 0 | 0 | 0 | 0 | 0 | 0 |
| 3 | D    | 1 | 0 | 0 | 0 | 0 | 1 |
| 4 | E    | 0 | 1 | 0 | 0 | 0 | 0 |
| 5 | F    | 0 | 1 | 0 | 1 | 0 | 0 |

邻接表

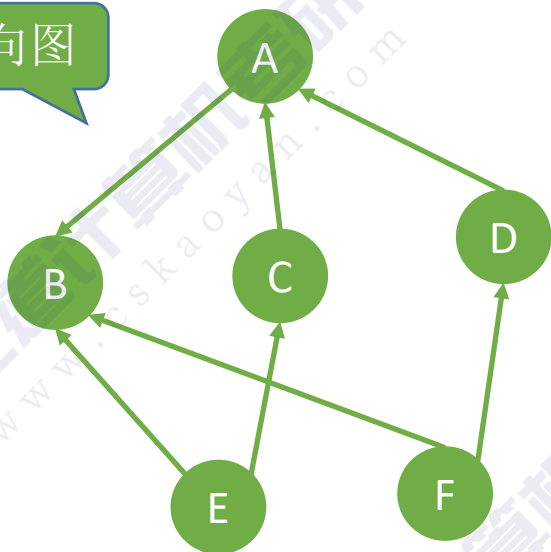
$O(1) \sim O(|E|)$

|   | data | *first      |
|---|------|-------------|
| 0 | A    | 1 → 3 ^     |
| 1 | B    | 0 → 4 → 5 ^ |
| 2 | 空    | ^           |
| 3 | D    | 0 → 5 ^     |
| 4 | E    | 1 ^         |
| 5 | F    | 1 → 3 ^     |

## 图的基本操作

- DeleteVertex( $G, x$ ): 从图 $G$ 中删除顶点 $x$ 。

有向图



邻接矩阵

$O(|V|)$

|   | data |   | A | B | C | D | E | F |
|---|------|---|---|---|---|---|---|---|
| 0 | A    | A | 0 | 1 | 0 | 0 | 0 | 0 |
| 1 | B    | B | 0 | 0 | 0 | 0 | 0 | 0 |
| 2 | C    | C | 1 | 0 | 0 | 0 | 0 | 0 |
| 3 | D    | D | 1 | 0 | 0 | 0 | 0 | 0 |
| 4 | E    | E | 0 | 1 | 1 | 0 | 0 | 0 |
| 5 | F    | F | 0 | 1 | 0 | 1 | 0 | 0 |

邻接表

|   | data | *first    |
|---|------|-----------|
| 0 | A    | → 1 ^     |
| 1 | B    | → ^       |
| 2 | C    | → 0 ^     |
| 3 | D    | → 0 ^     |
| 4 | E    | → 1 → 2 ^ |
| 5 | F    | → 1 → 3 ^ |

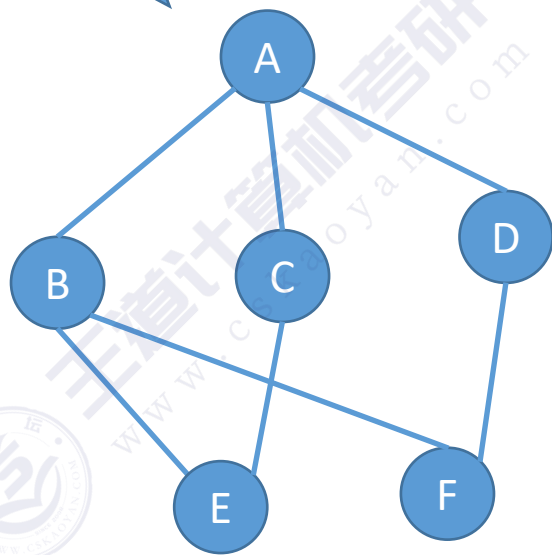
删出边:  $O(1) \sim O(|V|)$   
删入边:  $O(|E|)$

# 图的基本操作

- $\text{AddEdge}(G, x, y)$ : 若无向边 $(x, y)$ 或有向边 $\langle x, y \rangle$ 不存在, 则向图 $G$ 中添加该边。



无向图



邻接矩阵

$O(1)$

|   | data |   | A | B | C | D | E | F |
|---|------|---|---|---|---|---|---|---|
| 0 | A    | A | 0 | 1 | 1 | 1 | 0 | 0 |
| 1 | B    | B | 1 | 0 | 0 | 0 | 1 | 1 |
| 2 | C    | C | 1 | 0 | 0 | 0 | 1 | 0 |
| 3 | D    | D | 1 | 0 | 0 | 0 | 0 | 1 |
| 4 | E    | E | 0 | 1 | 1 | 0 | 0 | 0 |
| 5 | F    | F | 0 | 1 | 0 | 1 | 0 | 0 |

邻接表

$O(1)$

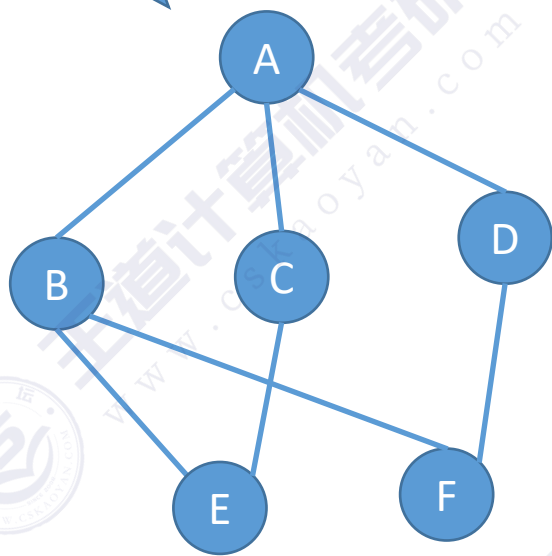
|   | data | *first      |
|---|------|-------------|
| 0 | A    | 1 → 2 → 3 ^ |
| 1 | B    | 0 → 4 → 5 ^ |
| 2 | C    | 0 → 4 ^     |
| 3 | D    | 0 → 5 ^     |
| 4 | E    | 1 → 2 ^     |
| 5 | F    | 1 → 3 ^     |

# 图的基本操作

- RemoveEdge(G,x,y): 若无向边(x, y)或有向边<x, y>存在, 则从图G中删除该边。



无向图



邻接矩阵

$O(1)$

|   | data |   | A | B | C | D | E | F |
|---|------|---|---|---|---|---|---|---|
| 0 | A    | A | 0 | 1 | 1 | 1 | 0 | 0 |
| 1 | B    | B | 1 | 0 | 0 | 0 | 1 | 1 |
| 2 | C    | C | 1 | 0 | 0 | 0 | 1 | 0 |
| 3 | D    | D | 1 | 0 | 0 | 0 | 0 | 1 |
| 4 | E    | E | 0 | 1 | 1 | 0 | 0 | 0 |
| 5 | F    | F | 0 | 1 | 0 | 1 | 0 | 0 |

邻接表

$O(1) \sim O(|V|)$

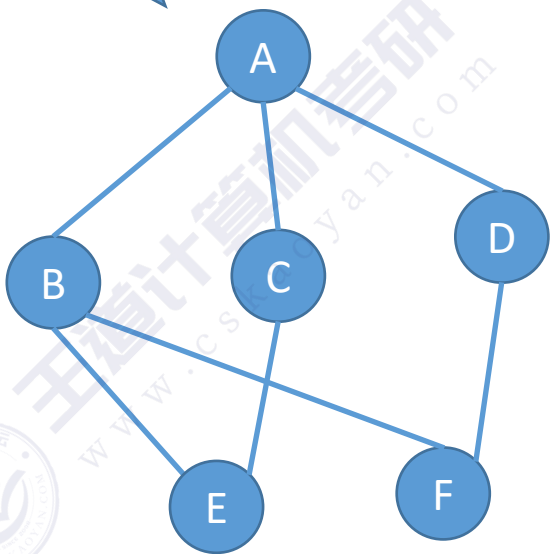
|   | data | *first      |
|---|------|-------------|
| 0 | A    | 1 → 2 → 3 ^ |
| 1 | B    | 0 → 4 → 5 ^ |
| 2 | C    | 0 → 4 ^     |
| 3 | D    | 0 → 5 ^     |
| 4 | E    | 1 → 2 ^     |
| 5 | F    | 1 → 3 ^     |



## 图的基本操作

- **FirstNeighbor(G,x)**: 求图G中顶点x的第一个邻接点, 若有则返回顶点号。若x没有邻接点或图中不存在x, 则返回-1。

无向图



邻接矩阵

$O(1) \sim O(|V|)$

|   | data |
|---|------|
| 0 | A    |
| 1 | B    |
| 2 | C    |
| 3 | D    |
| 4 | E    |
| 5 | F    |

|   | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A | 0 | 1 | 1 | 1 | 0 | 0 |
| B | 1 | 0 | 0 | 0 | 1 | 1 |
| C | 1 | 0 | 0 | 0 | 1 | 0 |
| D | 1 | 0 | 0 | 0 | 0 | 1 |
| E | 0 | 1 | 1 | 0 | 0 | 0 |
| F | 0 | 1 | 0 | 1 | 0 | 0 |

邻接表

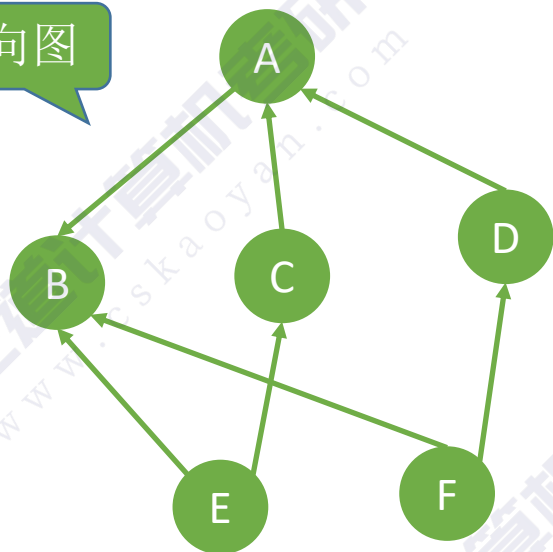
$O(1)$

|   | data | *first        |
|---|------|---------------|
| 0 | A    | → 1 → 2 → 3 ^ |
| 1 | B    | → 0 → 4 → 5 ^ |
| 2 | C    | → 0 → 4 ^     |
| 3 | D    | → 0 → 5 ^     |
| 4 | E    | → 1 → 2 ^     |
| 5 | F    | → 1 → 3 ^     |

## 图的基本操作

- **FirstNeighbor(G,x)**: 求图G中顶点x的第一个邻接点, 若有则返回顶点号。若x没有邻接点或图中不存在x, 则返回-1。

有向图



邻接矩阵

$O(1) \sim O(|V|)$

|   | data |   | A | B | C | D | E | F |
|---|------|---|---|---|---|---|---|---|
| 0 | A    | A | 0 | 1 | 0 | 0 | 0 | 0 |
| 1 | B    | B | 0 | 0 | 0 | 0 | 0 | 0 |
| 2 | C    | C | 1 | 0 | 0 | 0 | 0 | 0 |
| 3 | D    | D | 1 | 0 | 0 | 0 | 0 | 0 |
| 4 | E    | E | 0 | 1 | 1 | 0 | 0 | 0 |
| 5 | F    | F | 0 | 1 | 0 | 1 | 0 | 0 |

邻接表

找出边邻接点:  $O(1)$

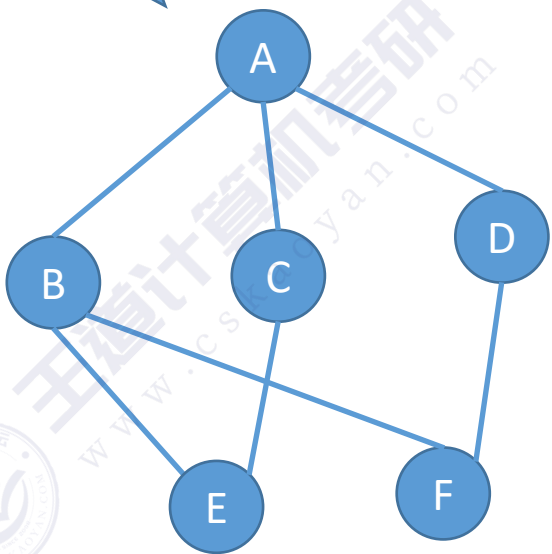
找入边邻接点:  $O(1) \sim O(|E|)$

|   | data | *first    |
|---|------|-----------|
| 0 | A    | → 1 ^     |
| 1 | B    | → ^       |
| 2 | C    | → 0 ^     |
| 3 | D    | → 0 ^     |
| 4 | E    | → 1 → 2 ^ |
| 5 | F    | → 1 → 3 ^ |

## 图的基本操作

- **NextNeighbor( $G, x, y$ ):** 假设图 $G$ 中顶点 $y$ 是顶点 $x$ 的一个邻接点, 返回除 $y$ 之外顶点 $x$ 的下一个邻接点的顶点号, 若 $y$ 是 $x$ 的最后一个邻接点, 则返回-1。

无向图



邻接矩阵

$O(1) \sim O(|V|)$

|   | data |
|---|------|
| 0 | A    |
| 1 | B    |
| 2 | C    |
| 3 | D    |
| 4 | E    |
| 5 | F    |

|   | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A | 0 | 1 | 1 | 1 | 0 | 0 |
| B | 1 | 0 | 0 | 0 | 1 | 1 |
| C | 1 | 0 | 0 | 0 | 1 | 0 |
| D | 1 | 0 | 0 | 0 | 0 | 1 |
| E | 0 | 1 | 1 | 0 | 0 | 0 |
| F | 0 | 1 | 0 | 1 | 0 | 0 |

邻接表

$O(1)$

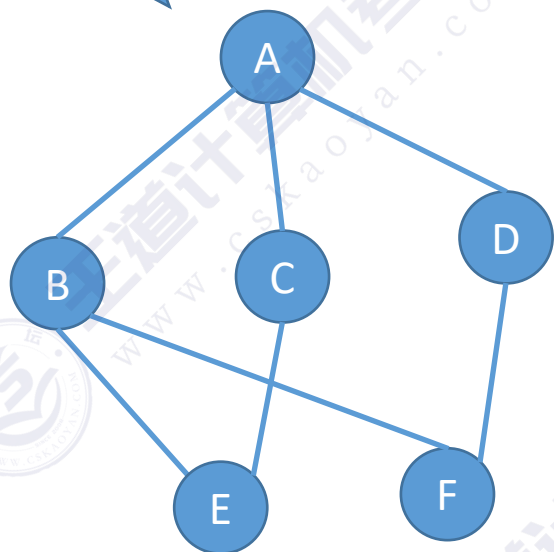
|   | data | *first              |
|---|------|---------------------|
| 0 | A    | → [1] → [2] → [3] ^ |
| 1 | B    | → [0] → [4] → [5] ^ |
| 2 | C    | → [0] → [4] ^       |
| 3 | D    | → [0] → [5] ^       |
| 4 | E    | → [1] → [2] ^       |
| 5 | F    | → [1] → [3] ^       |

# 图的基本操作

- $\text{Get\_edge\_value}(G,x,y)$ : 获取图 $G$ 中边 $(x,y)$ 或 $\langle x,y \rangle$ 对应的权值。
- $\text{Set\_edge\_value}(G,x,y,v)$ : 设置图 $G$ 中边 $(x,y)$ 或 $\langle x,y \rangle$ 对应的权值为 $v$ 。
- $\text{Adjacent}(G,x,y)$ : 判断图 $G$ 是否存在边 $\langle x,y \rangle$ 或 $(x,y)$ 。

雷同，核心在于找到边

无向图



邻接矩阵

$O(1)$

|   | data |   | A | B | C | D | E | F |
|---|------|---|---|---|---|---|---|---|
| 0 | A    | A | 0 | 1 | 1 | 1 | 0 | 0 |
| 1 | B    | B | 1 | 0 | 0 | 0 | 1 | 1 |
| 2 | C    | C | 1 | 0 | 0 | 0 | 1 | 0 |
| 3 | D    | D | 1 | 0 | 0 | 0 | 0 | 1 |
| 4 | E    | E | 0 | 1 | 1 | 0 | 0 | 0 |
| 5 | F    | F | 0 | 1 | 0 | 1 | 0 | 0 |

邻接表

$O(1) \sim O(|V|)$

|   | data | *first        |
|---|------|---------------|
| 0 | A    | → 1 → 2 → 3 ^ |
| 1 | B    | → 0 → 4 → 5 ^ |
| 2 | C    | → 0 → 4 ^     |
| 3 | D    | → 0 → 5 ^     |
| 4 | E    | → 1 → 2 ^     |
| 5 | F    | → 1 → 3 ^     |

## 知识回顾与重要考点

- $\text{Adjacent}(G, x, y)$ : 判断图 $G$ 是否存在边 $\langle x, y \rangle$ 或 $(x, y)$ 。
- $\text{Neighbors}(G, x)$ : 列出图 $G$ 中与结点 $x$ 邻接的边。
- $\text{InsertVertex}(G, x)$ : 在图 $G$ 中插入顶点 $x$ 。
- $\text{DeleteVertex}(G, x)$ : 从图 $G$ 中删除顶点 $x$ 。
- $\text{AddEdge}(G, x, y)$ : 若无向边 $(x, y)$ 或有向边 $\langle x, y \rangle$ 不存在, 则向图 $G$ 中添加该边。
- $\text{RemoveEdge}(G, x, y)$ : 若无向边 $(x, y)$ 或有向边 $\langle x, y \rangle$ 存在, 则从图 $G$ 中删除该边。
- $\text{FirstNeighbor}(G, x)$ : 求图 $G$ 中顶点 $x$ 的第一个邻接点, 若有则返回顶点号。若 $x$ 没有邻接点或图中不存在 $x$ , 则返回-1。
- $\text{NextNeighbor}(G, x, y)$ : 假设图 $G$ 中顶点 $y$ 是顶点 $x$ 的一个邻接点, 返回除 $y$ 之外顶点 $x$ 的下一个邻接点的顶点号, 若 $y$ 是 $x$ 的最后一个邻接点, 则返回-1。
- $\text{Get\_edge\_value}(G, x, y)$ : 获取图 $G$ 中边 $(x, y)$ 或 $\langle x, y \rangle$ 对应的权值。
- $\text{Set\_edge\_value}(G, x, y, v)$ : 设置图 $G$ 中边 $(x, y)$ 或 $\langle x, y \rangle$ 对应的权值为 $v$ 。

此外, 还有图的遍历算法, 包括深度优先遍历和广度优先遍历。